



Resumão de CSS



Status

Não iniciada

Até agora vimos que a página é feita com 3 "linguagens", HTML, CSS e JavaScript. E já vimos que o HTML é responsável por definir o que vai ter na nossa página, como textos, títulos, imagens, links, listas, tabelas, as divisões "<div>", entre outros. Hoje vamos entender como estilizar, ou deixar mais bonito essas tags, esses elementos, alterando cores, tamanhos, sombras, posicionamento, bordas e mais um monte de coisas. O que vamos usar para fazer essas alterações, é o CSS (**Cascading Style Sheets** ou Folha de Estilo em Cascata) que é uma linguagem de estilo, responsável por definir como o nosso documento html será apresentado. Nesse nome, "Folha de Estilo em Cascata", a folha de estilo, representa as regras para a estilização e "cascata" define qual será a ordem de prioridade na estilização, quem ganha de quem, por exemplo, se tivermos 2 estilos apontando para o mesmo elemento qual das duas regras será aplicada?

Então o CSS defini as regras é a prioridade da estilização no nosso documento html.



CSS

◆ Como adicionar CSS

Existem 3 formas de aplicar CSS no HTML:

1. Inline – direto na tag

```
<p style="color: green; font-size: 24px;">Texto Verde</p>
```

2. Interno – dentro do HTML

```
<head>
  <style>
    p { color: blue; }
  </style>
</head>
```

3. Externo – arquivo separado

```
<!-- HTML -->
<link rel="stylesheet" href="style.css">

/* style.css */
p { color: green; }
```

👉 Sempre que possível, use **externo**, pois facilita a manutenção.

◆ Diferença entre Classe e ID

- **Classe** (`.classe`) → pode ser usada em vários elementos.
- **ID** (`#id`) → deve ser único na página.

🧩 Exemplos

🎯 Classe

```
<p class="destaque">Texto em destaque</p>
<p class="destaque">Outro texto com o mesmo estilo</p>
```

```
.destaque {
  /* ".destaque" é uma classe.
     Isso significa que qualquer elemento HTML com class="destaque"
     vai receber esses estilos abaixo. */

  color: white; /* Define a cor do texto como branco. */
  background-color: #a243d2; /* Define a cor de fundo. Nesse caso, um roxo. */
  padding: 10px; /* Adiciona um espaço interno (entre o texto e a borda da caixa).
     É como se fosse uma "margem" dentro do elemento. */
  border-radius: 8px; /* Arredonda os cantos da caixa. Quanto maior o valor, mais arredondado. */
}
```



```
<h1 id="titulo-principal">Bem-vindo!</h1>
```

```
#titulo-principal {
  /* "#titulo-principal" é um ID.
     Ele aplica estilos apenas ao elemento HTML que tiver id="titulo-principal". */

  font-size: 40px;
  /* Define o tamanho da fonte (nesse caso, bem grande: 40 pixels). */

  text-align: center;
  /* Alinha o texto no centro da página ou do container. */

  color: #ff007f;
  /* Define a cor do texto em rosa choque (pode usar nomes de cores ou códigos hexadecimais). */
}
```

```
text-shadow: 2px 2px 5px black;
/* Adiciona uma sombra ao texto.
   - O primeiro valor (2px) desloca a sombra para a direita.
   - O segundo valor (2px) desloca a sombra para baixo.
   - O terceiro valor (5px) é o "borrão" da sombra.
   - O último valor (black) é a cor da sombra (preto). */
}
```

Guia de Bolso CSS

◆ Estrutura básica

```
<style>
  seletor {
    propriedade: valor;
  }
</style>
```

- **Seletor** → qual elemento do HTML será estilizado (`h1`, `p`, `.classe`, `#id`).
- **Propriedade** → o que você quer mudar (cor, tamanho, margem).
- **Valor** → como você quer que fique.

Cores e texto

- **color** → cor do texto.

```
p { color: red; } /* vermelho */
```

- **background / background-color** → cor de fundo.

```
body { background-color: #f4f4f9; }
```

- **font-family** → tipo da fonte.

```
body { font-family: Arial, sans-serif; }
```

- **font-size** → tamanho do texto.

```
h1 { font-size: 24px; }
```

- **font-weight** → espessura (negrito).

```
strong { font-weight: bold; }
```

- **text-align** → alinhamento.

```
h1 { text-align: center; }
```

As principais unidades (as que você mais vai usar)

px (pixel)

- Tamanho fixo
- Não muda com a tela

👉 Use quando quer **controle exato**

Exemplo falado: "margem de 16px"

% (porcentagem)

- Baseado no tamanho do elemento pai

👉 Use quando quer algo **proporcional**

Exemplo: margem que cresce ou diminui com o container

em

- Baseado no tamanho da fonte do elemento atual

👉 Se a fonte aumenta, a margem aumenta

Muito usada junto com texto

rem

- Baseado no tamanho da fonte do HTML (`html`)

👉 Mais previsível que `em`

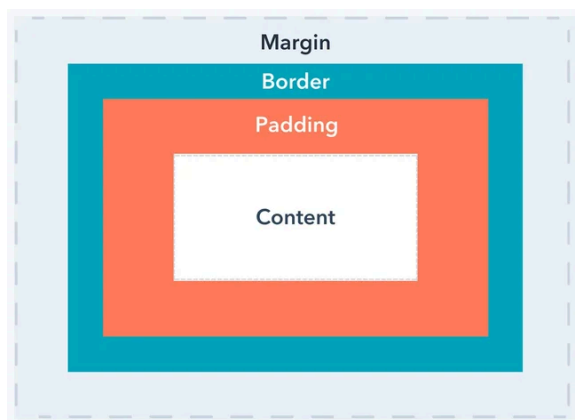
Muito comum em projetos modernos

vh / vw

- Baseado no tamanho da tela
- `vh` → altura da tela
- `vw` → largura da tela

👉 Use quando quer algo ligado ao **tamanho da tela**

📐 Espaçamentos



- **margin** → espaço externo (fora do elemento).

```
p { margin: 10px; } /* 20px em todos os lados */
```

👉 Resumindo para iniciantes:

- **1 valor** → todos os lados.
- **2 valores** → cima/baixo e esquerda/direita.

- **3 valores** → cima, lados, baixo.
- **4 valores** → cima, direita, baixo, esquerda (sentido horário).
- **padding** → espaço interno (dentro do elemento).

```
button { padding: 10px; }
```

- **border** → borda do elemento.

```
div { border: 1px solid black; }
```

- **border-radius** → arredonda bordas.

```
img { border-radius: 50%; } /* círculo */
```

Layout e posicionamento

- **width / height** → largura e altura.

```
div { width: 200px; height: 100px; }
```

display

O **display** define o **tipo de caixa** que um elemento gera e como ele se posiciona em relação aos outros.

◆ 1. **display: block;**

- O elemento ocupa **toda a largura disponível**.
- Quebra a linha (o próximo elemento fica embaixo).
- Exemplo: `<div>`, `<p>`, `<h1>` já são **block** por padrão.

```
div {  
  display: block;
```

```
}
```

```
<div>Sou um bloco</div>  
<div>Outro bloco</div>
```

→ Cada `div` fica em uma linha separada.

◆ 2. `display: inline;`

- O elemento ocupa **só o espaço do conteúdo**.
- Fica **na mesma linha** que outros inline.
- Exemplo: `<span>`, `<a>`, `<strong>` são **inline** por padrão.

```
span {  
  display: inline;  
}
```

```
<p>Texto <span>colorido</span> continua aqui.</p>
```

→ O `span` não quebra a linha.

◆ 3. `display: inline-block;`

- Mistura os dois: se comporta como inline (fica na mesma linha), **mas aceita altura, largura, padding e margin** como block.

```
button {  
  display: inline-block;  
  width: 120px;  
  height: 40px;  
}
```

→ Útil para botões lado a lado, mas com tamanho definido.

◆ 4. `display: none;`

- Esconde o elemento **completamente** (não ocupa espaço).
- Muito usado para **mostrar/esconder** algo com JavaScript.

```
#menu {  
  display: none;  
}
```

◆ 5. **display: flex;**

- Ativa o **Flexbox** (layout moderno).
- Permite alinhar e distribuir itens facilmente.

```
.container {  
  display: flex;  
  justify-content: center; /* centro no eixo horizontal */  
  align-items: center;      /* centro no eixo vertical */  
}
```

◆ 6. **display: grid;**

- Ativa o **CSS Grid** (layout em grade).
- Ótimo para fazer colunas e linhas.

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr; /* 3 colunas iguais */  
  gap: 10px;  
}
```

- **block** → ocupa toda a linha.
- **inline** → ocupa só o conteúdo.
- **inline-block** → lado a lado, mas aceita largura/altura.
- **none** → esconde o elemento.

- **flex** → layout flexível em linha/coluna.
 - **grid** → layout em grade.
-

Efeitos e decoração

- **box-shadow** → sombra na caixa.

```
div { box-shadow: 0 4px 6px rgba(0,0,0,0.2); }
```

- **text-shadow** → sombra no texto.

```
h1 { text-shadow: 2px 2px 4px gray; }
```

- **opacity** → transparência (0 a 1).

```
img { opacity: 0.7; }
```

- **hover** → efeito quando o mouse passa por cima.

```
button:hover { background: blue; }
```

- **cursor** → tipo do ponteiro.

```
button { cursor: pointer; }
```

Seletores úteis

- **p** → todos os parágrafos `<p>`.
 - **.classe** → todos os elementos com `class="classe"`.
 - **#id** → elemento único com `id="id"`.
 - **p span** → todo `<span>` dentro de `<p>`.
 - **p: hover** → parágrafo quando o mouse passar.
-

Dicas

- Sempre teste no navegador: altere e recarregue.
- Use **px** (pixels), **%** (percentual), **em/rem** (tamanho relativo).
- Comece com propriedades simples (cor, margem, fonte).
- Vá adicionando gradualmente novas propriedades conforme se sentir confortável.

Exemplos com efeitos visuais atrativos

Botão interativo

```
<button class="btn">Clique Aqui</button>
```

```
.btn {  
  /* ".btn" é uma CLASSE.  
   Todo elemento HTML com class="btn" vai ter esse estilo. */  
  
  background: linear-gradient(45deg, #a243d2, #ff007f);  
  /* Fundo em degradê (gradiente) na diagonal (45 graus),  
   indo do roxo (#a243d2) para o rosa (#ff007f). */  
  
  color: white;  
  /* A cor do TEXTO dentro do botão é branca. */  
  
  border: none;  
  /* Remove a borda padrão do botão. */  
  
  padding: 12px 24px;  
  /* Espaçamento interno (alto/baixo: 12px | lados: 24px).  
   Isso aumenta o tamanho do botão e dá mais espaço ao texto. */  
  
  font-size: 18px;  
  /* Define o tamanho do texto dentro do botão. */  
  
  border-radius: 30px;
```

```

    /* Arredonda bastante as bordas do botão, deixando-o com
    formato de "pílula". */

    cursor: pointer;
    /* Muda o cursor para a mãozinha quando passar por cima,
    mostrando que é clicável. */

    transition: 0.3s;
    /* Suaviza animações (hover) em 0,3 segundos. */
}

.btn:hover {
    /* Estilo aplicado apenas quando o mouse passa por cima d
    o botão. */

    transform: scale(1.1);
    /* Aumenta o tamanho do botão em 10% (efeito de zoom). */

    box-shadow: 0 0 15px #a243d2;
    /* Adiciona uma sombra ao redor do botão,
    criando um efeito de brilho (nesse caso, roxo). */
}

```

Cartão estilizado

```

<div class="card">
  <h2>Título</h2>
  <p>Conteúdo do card.</p>
</div>

```

```

.card {
    /* ".card" é uma CLASSE.
    Qualquer elemento HTML com class="card" vai ter esse e
    stilo. */

    background: white;

```

```

    /* Fundo branco para destacar o conteúdo dentro da caixa.
    */

    border-radius: 15px;
    /* Bordas arredondadas (15px) deixam a caixa com cantos s
    uaves. */

    padding: 20px;
    /* Espaçamento interno: cria “respiro” entre o conteúdo e
    as bordas. */

    width: 250px;
    /* Define a largura fixa do card (250px). */

    box-shadow: 0 4px 10px rgba(0,0,0,0.2);
    /* Adiciona sombra para dar profundidade:
    - 0 → deslocamento lateral (nenhum neste caso).
    - 4px → deslocamento vertical (a sombra fica para baix
    o).
    - 10px → espalhamento (borrão da sombra).
    - rgba(0,0,0,0.2) → cor preta com 20% de opacidade. */

    transition: 0.3s;
    /* Faz com que as mudanças (hover) sejam suaves. */
}

.card:hover {
    /* Estilos aplicados quando o mouse passa em cima do car
    d. */

    transform: translateY(-5px);
    /* Move o card 5px para cima, criando efeito de “flutua
    r”. */

    box-shadow: 0 10px 20px rgba(0,0,0,0.3);
    /* Deixa a sombra maior e mais forte, reforçando o efeito
    de destaque. */
}

```

☀ Texto com gradiente

```
<h1 class="gradiente">Codi Academy</h1>
```

```
.gradiente {  
  /* ".gradiente" é uma CLASSE.  
   Qualquer elemento com class="gradiente" vai receber es  
se estilo. */  
  
  font-size: 50px;  
  /* Deixa o texto bem grande (50 pixels). */  
  
  text-align: center;  
  /* Centraliza o texto dentro do elemento pai. */  
  
  background: linear-gradient(90deg, #a243d2, #ff007f, #00e  
5ff);  
  /* Cria um fundo em degradê (gradiente linear):  
   - 90deg → o degradê vai da esquerda para a direita.  
   - #a243d2 (roxo) → cor inicial.  
   - #ff007f (rosa) → cor do meio.  
   - #00e5ff (ciano) → cor final. */  
  
  -webkit-background-clip: text;  
  /* Esse truque "recorta" o fundo no formato do texto.  
   Assim, o degradê aparece apenas nas letras. */  
  
  -webkit-text-fill-color: transparent;  
  /* Deixa a cor do texto transparente,  
   permitindo que o degradê de fundo seja visível dentro  
das letras. */  
}
```